

Tadori v2.1

Frozen Technical Specification

Base concept plus accepted implementation corrections

July 2026

Document order

1. **Tadori v2: Technical Concept** - the approved product thesis, workflow, agent interface, visualization direction, prototype architecture, and original twelve-week plan.
2. **Tadori v2.1: Frozen Implementation Corrections** - the controlling amendments for storage, snapshots, graph-diff terminology, observed agent events, test semantics, validation, multi-resolution visualization, evaluation, privacy, executable SQLite migrations, rename/move coalescing, garbage collection, and revised gates.

Precedence

Where the implementation-corrections appendix conflicts with the original v2 concept, the v2.1 appendix controls.

The working name, TypeScript/JavaScript scope, agent-supervision wedge, six-tool MCP interface, ATLAS separation, Review/Explore modes, and twelve-week phase order are frozen.

No further specification revision should occur without a new defect report.

Tadori v2: Technical Concept

Revised after critique. Status: portfolio-quality research prototype with product potential.
Wedge: supervising coding agents on partially-understood TypeScript repos.

1. Final product thesis

One sentence. Tadori is a provenance-typed repository graph that a coding agent retrieves context through and a human reviews through, so the human can see exactly what the agent knew, ignored, and changed.

One paragraph. When Claude Code modifies a repo you only partially understand, you're reviewing a diff written by something whose research process you never saw. Tadori fixes the asymmetry. It indexes the repo into a graph where every relationship is tagged with how it was learned (compiler fact, heuristic, git history, documentation, human annotation) and serves that graph to the agent through MCP tools. Because retrieval goes through Tadori, every context request is logged, and the log becomes a visual coverage map: here's what the agent inspected before touching your auth code, here's what it never looked at, here's the exact dependency change the patch introduced, here's the boundary it crossed. Explore mode exists so you understand the map well enough to judge the coverage.

Technical paragraph. One SQLite-backed representation serves both consumers. Nodes are packages, files, symbols, routes, tests, and ADR documents; edges are typed relations (imports, calls, implements, tests, routes_to, documents, changed_with) each carrying an origin enum and a coarse confidence enum, plus file:line evidence pinned to a commit. The MCP layer answers six structural queries with evidence citations, token budgets, and explicit omission manifests, and writes a retrieval event per call. The viz layer renders a position-stable 2D base map and treats everything else (task focus, retrieval trace, planned scope, post-patch diff, decision pins) as overlays on cached coordinates. Post-patch, an incremental reindex produces two graph versions at two commits; the architecture diff is exact set arithmetic on edges, not prediction. The novelty claim rests on the combination: shared provenance-typed representation + retrieval logging as a byproduct of being the context provider + exact structural diffs, evaluated with a controlled 2D/2.5D/3D supervision study.

2. Primary workflow (Claude Code, session revocation example)

Repo: a Next.js + Express monorepo with `packages/web`, `packages/api`, `packages/core`. Sessions live in Redis behind `SessionStore`. Task: invalidate all of a user's sessions when they change their password.

1. **Index.** `tadori index` builds the graph at commit `a1b2c3`, ~40s cold for 150k LOC. `tadori serve` starts the MCP server and the viz at localhost:4114. A file watcher keeps a dirty set.
 2. **Task.** User tells Claude Code: "when a user changes their password, revoke all their other sessions." Tadori's MCP server is registered in the Claude Code config; a `SessionStart` hook posts the task description to Tadori, creating a task record.
 3. **Retrieval.** The agent calls `repo_overview`, then `find_symbol("session")`, then `symbol_context(anchor="SessionStore", relations=["calls", "implements", "tests"], depth=2, token_budget=3000)`, then `find_tests("PasswordService")`. Each response cites file:line@commit and flags two edges as `inferred` (dynamic `this[handler]()` dispatch in the auth middleware).
 4. **Trace.** Every call logs an event plus the node IDs returned. In the viz, retrieved nodes get a green fill; `packages/api/src/auth/refresh-token.ts` stays gray. That gray is the whole point.
 5. **Source inspection.** Tadori returned locations and key bodies; the agent reads the actual files with its own Read tool. Claude Code `PostToolUse` hooks forward Read events to Tadori, which maps paths to nodes and marks them inspected (weaker provenance than MCP retrieval, still shown).
 6. **Proposed edits.** The agent states its plan; a hook forwards it. Files named in the plan get a red outline. The user (or agent) calls `impact(["password-service.ts"]): 11 dependents, 2 boundary crossings, 4 linked tests. Rendered as a blast-radius overlay.`
 7. **Patch.** Agent edits `password-service.ts` and `session-store.ts`, adds `revokeAllForUser`.
 8. **Tests.** Agent runs the 4 linked tests plus the auth suite. Results attach to the task record.
 9. **Reindex.** The watcher saw 2 dirty files; incremental reindex of just those files and their import neighbors, ~1-2s. New graph version at commit `d4e5f6`.
 10. **Diff.** Edge set at `d4e5f6` minus edge set at `a1b2c3`: one new call edge `PasswordService.changePassword → SessionStore.revokeAllForUser` (green), no removed edges, no boundary violations. If the agent had imported `redis-client` directly from `packages/web`, the diff flags it against the declared boundary rule.
 11. **Review.** Human opens Review mode: attention overlay + diff + test results + the agent's plan side by side. They notice refresh-token rotation was never inspected but shares `SessionStore`. One click isolates that path in Explore mode; they read it, find the rotation path re-creates sessions after revocation, and send the agent back with a specific instruction. That catch is the demo.
-

3. Minimum repository representation

SQLite, WAL mode, better-sqlite3. Confidence is an enum, not a float nobody can calibrate.

sql

```

CREATE TABLE repos (
  id INTEGER PRIMARY KEY, root_path TEXT NOT NULL, created_at INTEGER
);

CREATE TABLE commits (
  id INTEGER PRIMARY KEY, repo_id INTEGER REFERENCES repos,
  sha TEXT NOT NULL, indexed_at INTEGER,
  UNIQUE(repo_id, sha)
);

CREATE TABLE files (
  id INTEGER PRIMARY KEY, repo_id INTEGER REFERENCES repos,
  path TEXT NOT NULL, package TEXT,
  content_hash TEXT NOT NULL,          -- hash at index time
  indexed_commit_id INTEGER REFERENCES commits,
  UNIQUE(repo_id, path)
);

CREATE TABLE nodes (
  id INTEGER PRIMARY KEY, repo_id INTEGER REFERENCES repos,
  kind TEXT CHECK(kind IN ('package', 'file', 'function', 'method', 'class',
    'interface', 'type', 'route', 'test', 'adr', 'doc_section')),
  name TEXT NOT NULL, qualified_name TEXT,
  file_id INTEGER REFERENCES files,    -- null for package nodes
  span_start INTEGER, span_end INTEGER,
  signature TEXT, exported INTEGER DEFAULT 0,
  indexed_commit_id INTEGER REFERENCES commits
);
CREATE VIRTUAL TABLE node_fts USING fts5(name, qualified_name, signature, path)

CREATE TABLE edges (
  id INTEGER PRIMARY KEY,
  src INTEGER REFERENCES nodes, dst INTEGER REFERENCES nodes,
  relation TEXT CHECK(relation IN ('contains', 'imports', 'exports', 'calls',
    'implements', 'extends', 'tests', 'routes_to', 'documents', 'changed_with')),
  origin TEXT CHECK(origin IN ('compiler', 'heuristic', 'git', 'doc', 'human', 'llm'
  confidence TEXT CHECK(confidence IN ('certain', 'likely', 'inferred')),
  indexed_commit_id INTEGER REFERENCES commits
);
CREATE INDEX idx_edges_src ON edges(src, relation);
CREATE INDEX idx_edges_dst ON edges(dst, relation);

CREATE TABLE evidence (

```

```

    id INTEGER PRIMARY KEY,
    edge_id INTEGER REFERENCES edges, node_id INTEGER REFERENCES nodes,
    file_id INTEGER REFERENCES files, line_start INTEGER, line_end INTEGER,
    commit_sha TEXT
);

CREATE TABLE decisions (
    id INTEGER PRIMARY KEY, repo_id INTEGER,
    kind TEXT CHECK(kind IN ('adr', 'doc', 'annotation')),
    title TEXT, status TEXT, source_file_id INTEGER REFERENCES files,
    body_excerpt TEXT
);

CREATE TABLE decision_links (
    decision_id INTEGER REFERENCES decisions,
    node_id INTEGER REFERENCES nodes,
    evidence_id INTEGER REFERENCES evidence,
    confidence TEXT CHECK(confidence IN ('certain', 'likely')) -- never 'inferred'
);

CREATE TABLE tasks (
    id INTEGER PRIMARY KEY, repo_id INTEGER, agent TEXT,
    description TEXT, started_at INTEGER, status TEXT
);

CREATE TABLE retrieval_events (
    id INTEGER PRIMARY KEY, task_id INTEGER REFERENCES tasks,
    tool TEXT, args_json TEXT, ts INTEGER,
    truncated INTEGER DEFAULT 0
);

CREATE TABLE retrieval_results (
    event_id INTEGER REFERENCES retrieval_events,
    node_id INTEGER REFERENCES nodes,
    stale INTEGER DEFAULT 0, omitted INTEGER DEFAULT 0 -- omitted = named in man
);

CREATE TABLE patches (
    id INTEGER PRIMARY KEY, task_id INTEGER REFERENCES tasks,
    base_commit_id INTEGER REFERENCES commits,
    head_commit_id INTEGER REFERENCES commits, ts INTEGER
);

CREATE TABLE patch_files (
    patch_id INTEGER REFERENCES patches, file_id INTEGER REFERENCES files,
    change_kind TEXT CHECK(change_kind IN ('added', 'modified', 'deleted'))
);

```

```
CREATE TABLE layout (
  node_id INTEGER REFERENCES nodes, view TEXT DEFAULT 'base',
  x REAL, y REAL, z REAL, pinned INTEGER DEFAULT 0,
  PRIMARY KEY(node_id, view)
);

CREATE TABLE summaries (  -- LLM output quarantined here, never in nodes/edges
  node_id INTEGER REFERENCES nodes, commit_id INTEGER REFERENCES commits,
  model TEXT, text TEXT, created_at INTEGER
);
```

Staleness. The watcher maintains an in-memory dirty set of paths whose live content hash differs from `files.content_hash`. Any query result touching a dirty file gets `stale: true`. After reindex, `content_hash` and `indexed_commit_id` update and the flag clears. No timestamps-as-truth, no guessing.

4. Extraction architecture

Comparison.

- **Tree-sitter.** Fast, error-tolerant, 100+ grammars. No type resolution, so calls and imports are name matching. Fine for structure, wrong tool for precise edges.
- **TypeScript compiler API.** Real module resolution and type checking, works on JS via `allowJs`/`checkJs`. Slower, needs a tsconfig, but this is a TS-first prototype and precision is the product.
- **SCIP (scip-typescript).** Precise defs/refs in a portable format, but it's built on the TS compiler anyway, and you'd still need your own passes for calls-with-enclosing-symbol, routes, and tests. Adopting the *format* later for export is cheap; adopting it as the engine now adds a layer.
- **Language servers.** Designed for interactive queries, awkward and slow for batch indexing. No.

Selection: TypeScript compiler API as the single engine. One AST, one resolver, fewer disagreement bugs. Use the LanguageService (not one-shot `createProgram`) so incremental updates reuse the program. Skip tree-sitter in v1; revisit only if plain-JS-heavy repos or indexing speed force it. Keep node/edge shapes mappable to SCIP for a future export.

Extraction passes and their honesty labels.

What	How	Origin / confidence
Imports/exports	checker module resolution	compiler / certain
Definitions, classes, interfaces	AST + symbol table	compiler / certain
References	checker <code>findReferences</code> -equivalent walk	compiler / certain
Calls (resolved)	call expression → resolved signature → declaration; enclosing function found by span containment	compiler / certain
Calls (unresolved: <code>any</code> , dynamic <code>obj[k]()</code> , DI)	name match within import scope	heuristic / inferred
Express routes	<code>app.get/post/...</code> and <code>router.METHOD</code> with literal paths, handler resolution via checker	compiler / certain (literal), heuristic / likely (computed paths)
Next routes	filesystem convention (<code>app/</code> , <code>pages/</code> , route handlers)	compiler / certain
Tests	filename patterns + <code>describe/it/test</code> calls; <code>tests</code> edges from test file imports/calls into targets	compiler / certain (linkage), heuristic / likely (naming-only)
Package boundaries	package.json workspaces + directory containment; declared rules from a <code>tadori.rules.json</code> (dependency-cruiser-style allow/deny)	compiler / certain
Git history	<code>git log --numstat</code> ; <code>changed_with</code> edges where co-change count ≥ 5 and lift over baseline	git / likely
ADR/doc links	parse <code>docs/**/*.md</code> and <code>adr/</code> ; markdown links and backtick path references that resolve to a real file/symbol → certain; bare symbol-name mentions with unique match → likely; ambiguous mentions dropped	doc / certain or likely

Expected weak spots, documented up front: dynamic dispatch, dependency injection containers, `require()` with computed paths, and JSX component usage as "calls" (model component render as a `calls` edge, heuristic). Sample and report call-edge precision on fixture repos rather than asserting it.

5. Six-tool MCP API

Common behavior for all six: every returned item carries `{file, line_start, line_end, commit}` evidence and a `stale` flag; every edge carries `{origin, confidence}`; every call writes one `retrieval_events` row plus `retrieval_results` rows; token budgets are enforced by ranked truncation with an **omission manifest** (`omitted: [{node, reason, score}]`, top 10 by rank) so nothing is silently dropped; `truncated: true` plus a `cursor` lets the agent page rather than assume completeness. Evidence selection beats compression: when over budget, degrade bodies → signatures → names before dropping nodes.

1. `repo_overview()` Purpose: orient once per task. Returns packages with LOC and dependency direction, entry points, route table, declared boundary rules, top-level directory roles, counts. No bodies. Budget: fixed small (~1.5k tokens); truncates route table first with manifest. Logged with zero node results at symbol level, package nodes only.
2. `find_symbol(query, kind?, limit=10)` Purpose: locate by name/signature/path. FTS5 + exact-match boost. Returns matches with kind, qualified name, signature, location, fan-in. Stale matches flagged. Truncation: `limit`, remainder count reported.
3. `symbol_context(anchor, relations[], depth=1..2, token_budget)` Purpose: the workhorse. Returns anchor body (if it fits), neighbor signatures grouped by relation, edges with provenance, linked decisions, linked tests. Ranking per §6, hard-include guarantees enforced before budget applies. If hard-include set alone exceeds budget: signatures-only for everything, no bodies, and say so.
4. `find_tests(target)` Purpose: which tests exercise this node, via `tests` edges (import/call linkage first, naming heuristic second, labeled). Returns test file, test names when parseable, run-command hint from package.json scripts. If empty: says "no linked tests found," never guesses.
5. `impact(targets[] | diff, depth=1..3)` Purpose: blast radius. Transitive dependents over `calls`+`imports`, boundary crossings against declared rules, affected tests, fan-in per hop. Accepts a unified diff and maps hunks to nodes by span. Depth-limited; beyond-depth dependents summarized as counts per package with manifest.
6. `path(from, to, relations=["calls","imports"], k=3)` Purpose: how does A reach B. Up to k shortest paths, each edge with provenance so the agent can see when a hop is inferred. No path: returns nearest-approach nodes instead of nothing.

6. Retrieval ranking

Linear score, hand-set weights, no ML infrastructure:

```
score(n) = 3.0·bm25(task_text, n)      -- FTS5 over name/signature/path
        + 2.5·prox(n)                  -- 1/(1+graph_distance) from
anchors
        + 1.0·ln(1+fan_in(n))
        + 1.0·churn(n)                  -- commits touching n in last 90d,
capped
        + 1.5·[n has tests edge to anchor set]
        + 1.0·[n linked to a decision]
        + 0.5·[same package as anchor]
```

Weights live in one config file and get tuned by running the §9 benchmark, not by feel.

The critical-neighbor guarantee. Ranking never gets to drop these; they're included before the budget is spent, as signatures at minimum:

- direct callers and callees (depth 1) of every anchor;
- type/interface definitions appearing in any anchor signature;
- tests with a `certain` linkage to any anchor;
- any depth-1 neighbor across a declared boundary (these are exactly the ones agents break).

Then ranked nodes fill the remaining budget, then the omission manifest names the next ten. An agent that reads "omitted: refresh-token.ts (score 4.2, reason: budget)" can go get it. An agent that never learns the file existed cannot. That asymmetry is the whole design argument for manifests over aggressive compression.

7. Visual system

Stable 2D base map (default, load-bearing). Sigma.js/WebGL. Layout: seeded force-directed run once at first index, positions written to `layout`, then frozen. New nodes on reindex are placed near their package centroid with local relaxation only; nothing else moves. Package convex hulls with labels. Semantic zoom, three levels exactly: packages → files → exported symbols. Edge rendering is the provenance legend and it never changes meaning: solid = compiler-certain, dashed = likely, dotted = inferred. Doc/git origins get distinct muted colors.

Overlays (all on the same frozen coordinates, composable, toggleable):

- *Task focus*: non-relevant nodes dimmed to 15% opacity; nothing moves.
- *Retrieval trace*: green fill on MCP-returned nodes, lighter green on hook-observed file reads, gray = never touched. Optional sequence numbers. A coverage stat in the corner: "agent inspected 14 of 31 nodes within blast radius."
- *Planned scope*: red outline on files the agent's plan names.
- *Post-patch diff*: added edges green, removed red, boundary violations get a badge and a shake-free (static) warning glyph.
- *Decision pins*: small bookmark glyph on nodes/hulls with `decision_links`; click → ADR text and evidence in the side panel.

2.5D layer (optional, one depth field at a time). Fixed-tilt orthographic view, no free camera. Depth is bound to exactly one queryable field: architectural layer (routes above services above data), or base-vs-patched (two planes, diff edges bridging them), or repository-vs-agent-scope. If a depth assignment can't be expressed as a SQL expression over the schema, it's not allowed.

Experimental 3D (research branch, off by default). Same graph, z bound to a field (layer, retrieval order, or commit time), free orbit permitted only here. Exists to be measured in §8, not to demo. No buildings, no terrain, no particles.

Side panels, not graph: source code, evidence lists with file:line links, ADR bodies, retrieval log table, omission manifests, test results, task history. The graph shows topology and coverage; text lives in text.

Anti-decoration rule. Every visual channel (position, size, color, fill, outline, depth, badge) maps to a named column or query. Right-click any encoding → "show the rows." If an encoding can't answer that, it gets deleted. This rule is written into the repo's own `tadori.rules.json` as a reminder to future me.

8. Visual evaluation experiment

Design. Within-subjects, 4 conditions, Latin-square ordered across 4 matched task sets on 2 unfamiliar repos (~80-150k LOC each, e.g. Cal.com and a mid-size Express app): (1) IDE only, (2) IDE + conventional dependency graph (dependency-cruiser SVG or Madge output), (3) IDE + Tadori 2D, (4) IDE + Tadori 2.5D/3D. Participants: 12-16, recruited from TAMU CS/ECE upper-division and local devs, screened for TS experience. 60-75 min sessions, screen-recorded.

Tasks per condition (one each): trace the authentication flow end to end and name every module it crosses; given a proposed change to symbol X, list the tests that must run; find the

seeded boundary violation; review a prepared agent-generated patch and accept/reject with justification; state which areas the agent failed to inspect before that patch (Tadori conditions only for this last one; in non-Tadori conditions participants get the raw tool-call log as text, which is the honest baseline).

Measures. Correctness (rubric-scored by two graders), time to answer, navigation errors (wrong-file opens from screen recording), seeded violations detected, review confidence (calibrated: confidence vs actual correctness), NASA-TLX after each condition, 48-hour retention quiz (5 structural questions per repo), preference ranking last and reported as secondary only.

3D removal criteria, stated in advance. Cut 3D/2.5D-with-orbit if any of: (a) not significantly better than Tadori 2D on at least one of the two spatial task families (boundary finding, multi-hop tracing) at $p < .05$ or a medium effect size given the small n ; (b) worse than 2D on time or TLX by more than 15%; (c) more than a quarter of participants report disorientation or motion discomfort. 2.5D fixed-tilt survives on a lower bar (no significant harm plus a directional win on the diff-review task) since it costs little. Preference alone saves nothing.

9. Agent supervision experiment

Conditions. (1) Plain Claude Code with its native search. (2) Claude Code + an existing code-graph MCP server (codegraph or codebase-memory-mcp) as the real baseline, not a strawman. (3) Claude Code + Tadori MCP.

Corpus. 3 TS repos, 50-150k LOC, plus 24-30 tasks: feature additions, bug fixes with misleading symptom locations, refactors, and 6-8 tasks with seeded traps (a boundary that must not be crossed, a shared dependency with a non-obvious second consumer, a stale-looking but load-bearing module). Each task has a held-out test suite defining success.

Agent-side metrics. Task success (held-out tests pass), regressions (previously passing tests broken), unrelated files changed, relevant files inspected before edit (against a per-task gold set), boundary violations introduced, unsupported claims (statements in the agent's final summary not backed by any retrieved or read source, hand-audited), input+output tokens, tool calls, wall time. Token reduction is reported but is explicitly not a win condition on its own; a token win with a success-rate loss is scored as a loss, which is exactly the trade the Codebase-Memory paper's own numbers showed (83% vs 92% quality at 10x fewer tokens).

Human-side sub-experiment (does the supervision layer catch errors). Take 12-16 agent-generated patches from the runs, half containing a seeded or naturally occurring defect (boundary violation, missed second consumer, untested path). Reviewers assess each patch twice-crossed: with the standard diff + agent transcript, and with the Tadori review screen

(trace + diff + manifest). Measure defect detection rate, false accepts, time to decision, and confidence calibration. The layer earns its existence if detection rate is at least 1.5x the transcript-only condition; the target story is "reviewers caught twice as many agent mistakes."

10. Prototype architecture and packages

pnpm monorepo, local-first, no cloud anything.

tadori/	
packages/core	graph types, relation/origin enums, shared zod schemas
packages/indexer	TS LanguageService driver, extractors (routes, tests, git, docs), incremental pipeline
packages/store	better-sqlite3, migrations, query layer, FTS
packages/rank	scoring, hard-include logic, omission manifests
packages/mcp	@modelcontextprotocol/sdk server, six tools, event log
packages/hooks	Claude Code hook receivers (task start, plan, file reads)
packages/cli	tadori index serve diff check
packages/server	thin HTTP+WS API for the viz (fastify)
apps/viz	React + Sigma.js; R3F behind an /experiment flag only
packages/fixtures	3 golden repos with hand-verified expected graphs
packages/bench	task runner, seeded-trap repos, metrics collection

Key deps: typescript, better-sqlite3, @modelcontextprotocol/sdk, fastify, react, sigma, graphology (layout), simple-git, chokidar, zod, vitest. Three.js/R3F stays quarantined in the experiment flag so the default bundle never pays for it.

11. Twelve-week sequence

- **W1-2 — Store + core extraction.** Schema, migrations, LanguageService driver, imports/exports/definitions/references on fixture repos. Gate: golden-graph snapshot tests pass; full index of a 150k LOC fixture under 60s.
- **W3 — Semantic edges.** Resolved calls with enclosing-symbol attribution, Express/Next routes, test linkage, git churn, provenance tagging throughout. Gate: sampled call-edge precision ≥85% on fixtures (hand-check 100 edges).
- **W4 — MCP.** Six tools, evidence citations, stale flags, event logging. Register with Claude Code, start dogfooding on Icon Dock and the portfolio site repo. Gate: an end-to-end Claude Code task completes using only Tadori tools for structure queries.

- **W5 — Ranking + budgets.** Scoring, hard includes, omission manifests, cursors. Micro-benchmark: 10 tasks, Tadori vs codegraph-class baseline, measure relevant-file recall in returned context. Gate: recall $\geq$ baseline with manifests present in 100% of truncated responses.
- **W6 — Incremental + hardening.** Watcher, dirty set, incremental reindex, `tadori diff`. Perf pass to 250k LOC. Gate: incremental reindex of a single-file change under 2s p95; document where it breaks.
- **W7-8 — 2D base map.** Layout + freeze, hulls, semantic zoom, provenance edge styles, task-focus and retrieval-trace overlays, side panels wired to store. Gate: load a cold 150k LOC repo to interactive map in under 5s; positions byte-identical across reloads.
- **W9 — Review mode.** Post-patch diff overlay, boundary rules + violation badges, planned-scope overlay from hooks, ADR/doc extraction + decision pins. Gate: on 20 real merged PRs from fixture repos, edge-diff reports match hand-derived truth 100%.
- **W10 — Depth experiment build.** 2.5D fixed-tilt with layer and base-vs-patched bindings; 3D branch behind flag. Timeboxed hard: whatever exists Friday is what gets tested. Gate: both render the same store queries as 2D, zero bespoke data paths.
- **W11 — Supervision benchmark.** Run §9 across all three conditions, hand-audit unsupported claims, run the reviewer sub-experiment on collected patches. Gate: data collected, whatever it says.
- **W12 — Visual study + writeup.** Run §8 with 12-16 participants, analyze, write the report/paper draft, cut the demo video (the session-revocation walkthrough from §2). Gate: results written down, including negative ones.

Visual polish is never a milestone. If a week slips, the depth experiment (W10) shrinks before anything else does.

12. Strict exclusions

Not in this prototype, no exceptions without a written reason: universal language support beyond TS/JS; runtime tracing or instrumentation; inferred design rationale presented as fact (hypothesis pipeline is a post-W12 experiment, human-confirmed before it ever becomes an annotation); cloud sync or collaboration; target-architecture editing or aspirational-architecture diffing; predicted ghost topologies from natural-language plans (only exact post-patch diffs); multi-agent orchestration or any agent framework of our own; IDE replacement features (no editing in Tadori, ever); any build or runtime dependency on ATLAS in either direction. The optional Tadori-to-Dossier export stays a design note.

Written down now so the results can't be lawyered later.

- **Graph retrieval adds no value** if condition 3 in §9 trails plain Claude Code by >3 points task success, or matches it without reducing unrelated-file edits or missed-consumer errors. Response: kill the retrieval layer, pivot to transcript-ingestion-only observability.
 - **Attention visualization adds no value** if reviewer defect detection with the Tadori screen is under 1.5x the transcript baseline, or reviewers ignore the overlay in recordings. Response: keep the diff, drop the map.
 - **Decision linking adds no value** if link precision <90% on fixture ADRs or pins go unclicked across all dogfooding and study sessions. Response: cut to a plain "docs referencing this file" list.
 - **3D adds no value** per the §8 criteria. Response: delete the branch, report the negative result (which is itself a citable finding).
 - **Portfolio-only ceiling** if everything works but wins are marginal: agent metrics within noise of baselines and reviewers only modestly better. Then it's a strong demo + writeup, not a product seed, and that's a fine outcome to declare honestly.
 - **Stop entirely** if either foundation fails: incremental reindex can't stay under ~2s p95 (stale graph = actively harmful, the one sin the concept can't survive), or sampled call-edge precision can't reach ~85% on ordinary TS code, meaning agents and humans alike will learn to distrust the map.
-

14. Name review

Tadoru (辿る) means to follow, trace, or pursue a path, as in *michi o tadoru* (trace a route) or *kioku o tadoru* (trace a memory); *tadori* is its noun/stem form, familiar from *tadoritsuku* (to finally reach). The meaning maps directly onto path tracing, retrieval traces, and provenance, which is better semantic alignment than most tool names get. Pronunciation is easy internationally: tah-DOH-ree, three open syllables, no ambiguous clusters. Conflict check: no developer tool, product, or notable project surfaced under the name; the only hit of note is a random GitHub username (Tadori-izanai). Grab the npm scope and GitHub org early to be safe. One honest caveat: the name is opaque to anyone who doesn't get the one-line etymology, so the tagline does real work ("Tadori: trace what your agent traced"). Retained.

15. Final recommendation

Build: Tadori, an agent supervision layer on a provenance-typed repository graph.

- **Primary user:** a developer supervising Claude Code (secondarily Codex) on TypeScript repositories they only partially understand, starting with the developer's own repos.
- **Primary workflow:** index → agent retrieves structure through six MCP tools → human reviews the retrieval trace, exact architecture diff, and boundary checks before accepting the patch. Explore mode exists to make Review mode legible.
- **Technical contribution:** a shared repository representation where every relationship carries origin and coarse confidence with pinned evidence; retrieval observability obtained as a byproduct of being the context provider rather than by instrumenting the agent; omission manifests as an alternative to silent compression; exact post-patch structural diffs from incremental reindexing.
- **Visual contribution:** a position-stable base map where tasks, agent coverage, and diffs are overlays instead of relayouts, plus the first controlled comparison of 2D, 2.5D, and 3D specifically for agent-supervision tasks, with pre-registered removal criteria for depth.
- **Agent contribution:** six evidence-citing MCP tools benchmarked against real existing graph servers, scored on correctness and inspection coverage first and tokens second.
- **Exact proof required at week 12:** (1) on a 24-30 task benchmark across 3 repos, Tadori matches or beats plain Claude Code on task success while reducing unrelated-file edits and missed-dependency errors, and beats an existing code-graph MCP baseline on relevant-file recall and evidence accuracy; (2) reviewers using the Tadori review screen catch at least 1.5x the agent-introduced defects of transcript-only review; (3) the 2D/2.5D/3D study produces a defensible answer either way. Hit (1) and (2) and this is a publishable dev-tools contribution and a real product seed. Miss them and §13 says exactly what to do about it.

The one experimental feature deliberately kept despite prior art doing weaker versions: the depth/3D comparison. Earlier tools bolted 3D on as decoration and never measured it; measuring it under pre-registered kill criteria is the difference between repeating their mistake and generating the evidence they never did.

Tadori v2.1

Frozen Implementation Corrections

Frozen specification package

July 2026

Table of Contents

1 Tadori v2.1 - Frozen Implementation Corrections

Status: Frozen and accepted

Base document: *Tadori v2: Technical Concept*

Rule: Where this document conflicts with Tadori v2, this document controls.

1.1 Locked decisions

- Working name: Tadori.
- Initial scope: TypeScript and JavaScript only.
- Primary workflow: supervising Claude Code, with Codex support through the same MCP-oriented core where practical.
- Product modes: Review mode is primary; Explore mode exists to make Review mode understandable.
- Exactly six initial MCP tools: `repo_overview`, `find_symbol`, `symbol_context`, `find_tests`, `impact`, and `path`.
- Stable 2D is the default view. 2.5D and 3D are experiments with removal criteria.
- No runtime tracing in the prototype.
- No undocumented design rationale presented as fact.
- No dependency on ATLAS.
- The product compares extracted graph snapshots; it does not claim complete runtime architecture truth.

1.2 1. Stable entities and snapshot memberships

Tadori uses stable entities plus snapshot-membership rows. It does not overwrite one mutable graph and it does not model every state as a Git commit.

Supported repository-state kinds:

- `commit`
- `working_tree`

- staged
- patch

A snapshot records its repository, kind, base commit when available, deterministic workspace hash, parent snapshot, creation time, and pin status.

1.2.1 Stable identities

Canonical identities use printable pipe separators. Before joining fields, escape backslashes and then pipes.

- File: file | <normalized path>
- Node: node | <kind> | <qualified name>
- Edge: edge | <source node key> | <relation> | <destination node key>

Canonical identities are UTF-8 encoded and SHA-256 hashed. Collision handling compares the full canonical identity; if a true collision occurs, a collision index is appended and the key is rehashed.

Node identity remains based on (kind, qualified_name). Edge identity remains based on (source key, relation, destination key).

1.2.2 Snapshot memberships

Per-snapshot rows carry state-specific data such as:

- path and content hash;
- package;
- source spans and lines;
- signature and body hash;
- edge provenance, confidence, and resolution;
- analyzer version;
- evidence locations.

This preserves stable references for layout positions, retrieval events, annotations, decisions, and task records while allowing multiple repository states to coexist.

1.3 2. Correct graph-diff language

Permitted claim:

Tadori computes the exact set difference between two extracted graph snapshots.

Prohibited claim:

Tadori computes the complete or exact change in the repository's real runtime architecture.

Diff classes:

- Added compiler-resolved relationship
- Removed compiler-resolved relationship
- Added likely extracted relationship
- Removed likely extracted relationship
- Added inferred relationship - verify in source
- Analyzer no longer infers this relationship
- Unresolved target
- Relationship resolution strengthened
- Relationship resolution weakened
- Moved or renamed - likely
- Possible move or rename - ambiguous

Compiler-derived, likely, inferred, unresolved, and property-change results must remain visually and textually distinct.

1.4 3. Observed repository attention

Tadori records observable repository interactions. It does not expose chain of thought or claim complete knowledge of the agent's context.

Observed event types:

- mcp_returned
- file_read_observed
- plan_mentioned
- modified
- test_selected
- test_executed
- capture_interrupted

The default state is **not observed inspected**. This means only that Tadori did not observe inspection through a supported MCP call, hook, imported transcript, or recognized tool event during that task.

Do not say:

- the agent ignored this;
- the agent did not know this;
- the agent never saw this.

Task-level observation coverage is one of:

- complete_for_registered_sources
- partial
- unknown

Package-level overlays must aggregate child events fractionally. A package with one observed child among two hundred is never painted as fully inspected.

1.5 4. Test linkage is not test coverage

Tadori distinguishes:

- statically linked test;
- naming-associated test;
- package-associated test;
- historically associated test;
- executed test;
- passed or failed test;
- runtime-covering test, unavailable in v1.

find_tests must say **Likely relevant tests**. Static linkage does not prove behavioral coverage.

1.6 5. Graph-quality validation

Precision and recall use different ground-truth methods.

1.6.1 Precision on real repositories

Use stratified random samples of emitted relations. Report two-sided 95% Wilson confidence intervals. Ambiguous reviewer judgments count as incorrect for release gating.

For a perfect sample, the Wilson lower bound is:

$LB = 1 / (1 + z^2 / n)$, where $z = 1.959964$.

Therefore:

- $LB \geq 0.98$ requires at least 189 perfect observations; use 200.
- $LB \geq 0.99$ requires at least 381 perfect observations; use 400.

1.6.2 Corrected precision and fixture-recall table

Relation	Real audit n	Required 95% Wilson lower bound	Maximum accepted errors	Fixture recall gate
Imports	200	≥ 0.98	0	≥ 0.98
Exports/re-exports	200	≥ 0.98	0	≥ 0.98
Definitions	400	≥ 0.99	0	≥ 0.99
References	200	≥ 0.97	1	≥ 0.95
Resolved calls	200	≥ 0.90	11	≥ 0.80
Heuristic calls	200	≥ 0.70	47	no global recall claim

Relation	Real audit n	Required 95% Wilson lower bound	Maximum accepted errors	Fixture recall gate
Express routes	200	≥ 0.95	3	≥ 0.90
Next.js routes	200	≥ 0.98	0	≥ 0.95
Compiler-linked tests	200	≥ 0.90	11	≥ 0.85
Naming-associated tests	150	≥ 0.75	27	no release recall gate
Exact ADR path links	200	≥ 0.95	3	≥ 0.90
Unique-symbol ADR links	200	≥ 0.90	11	≥ 0.80

If a stratum population is smaller than its target sample, audit the entire population as a census. High-bar relations with any false positive fail that census.

1.6.3 Recall

Recall is claimed only on synthetic or manually constructed fixtures whose complete expected graph is known by construction. Do not publish real-repository recall numbers without a complete audit.

1.6.4 Required relation metrics

Report separately:

- precision;
- fixture recall;
- F1 where both exist;
- unresolved candidate rate;
- excluded dynamic call count.

1.7 6. Multi-resolution visualization

Do not render one all-symbol graph.

Three levels:

1. Package aggregate - always resident.
2. File subgraph - loaded per selected package or task region.
3. Symbol task region - loaded only around anchors, direct relations, linked tests, paths, and bounded impact regions.

Positions are cached per level. Existing unrelated regions do not move during ordinary reindexing. New entities use local placement and bounded relaxation.

Cross-level edges terminate at aggregate portals until expanded. Aggregated counts must reconcile with the underlying edges.

The default interface remains stable 2D. Optional 2.5D binds depth to exactly one queryable field. Experimental 3D remains off by default and uses the same graph queries.

Anti-decoration rule:

Every visual channel must map to a named stored or computed field and must expose the underlying rows.

1.8 7. Evaluation sequence

1.8.1 During the 12-week build

Evaluate engineering properties without external participants:

- extraction quality;
- fixture recall;
- sampled precision;
- retrieval quality;
- task success;
- unrelated modifications;
- missed consumers;
- stale-result behavior;
- incremental indexing;
- extracted graph-diff correctness.

1.8.2 Post-build pilot

Use 12-16 participants to estimate effect sizes, variance, usability failures, disorientation, and task quality. This pilot is not a significance test.

Navigation and tracing tasks may be within-subject using different repos and counterbalancing. Patch review must be between-subject because seeing the same defect twice contaminates the comparison.

1.8.3 Formal study

Use pilot variance and power analysis. Provisional targets:

- approximately 40 recruits for within-subject navigation/tracing;
- 48-64 participants for repeated unique patch-review tasks analyzed with mixed effects.

1.8.4 Minimum useful depth effects

A depth-based condition must produce at least one of:

- = 8 percentage-point accuracy improvement with $\leq 10\%$ time increase and ≤ 5 NASA-TLX point increase;
- = 10% time reduction while remaining within a -3 percentage-point accuracy noninferiority margin;
- = 10 percentage-point defect-detection improvement without increasing false rejection by more than 5 points.

Free-orbit 3D is removed if it is materially slower, less accurate, more cognitively demanding, disorienting, frequently abandoned for 2D, or requires bespoke semantics outside the common query model.

1.9 8. Privacy and local operation

Default bind address: 127.0.0.1.

Apply the union of:

- .gitignore
- .tadoriignore
- built-in generated, binary, and large-file exclusions

Default exclusions include node_modules, build output, coverage, caches, maps, minified JavaScript, binaries, and large generated artifacts.

Redaction families include:

- AWS access keys;
- GitHub tokens;
- Google API keys;
- Slack tokens;
- generic sk- keys;
- JWTs;
- private-key blocks;
- .env values whose keys indicate secrets, passwords, tokens, API keys, private keys, database URLs, or connection strings.

Support:

- source-body suppression;
- excerpt-size limits;
- separate retention for snapshots, retrieval events, hook events, plans, test output, and excerpts;
- explicit repository configuration and consent;
- purge commands for tasks, logs, source bodies, snapshots, and complete repositories.

1.10 9. Executable SQLite schema

The implementation uses ordered migrations. The schema below is the accepted structural contract; application code calculates canonical hashes.

1.10.1 Migration 001 - repositories, snapshots, stable entities, memberships, evidence

PRAGMA foreign_keys = **ON**;

PRAGMA journal_mode = WAL;

PRAGMA synchronous = **NORMAL**;

```
CREATE TABLE IF NOT EXISTS schema_migrations (  
  version INTEGER PRIMARY KEY,  
  applied_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
BEGIN IMMEDIATE;
```

```
CREATE TABLE repositories (  
  id INTEGER PRIMARY KEY,  
  root_path TEXT NOT NULL UNIQUE,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE repository_snapshots (  
  id INTEGER PRIMARY KEY,  
  repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,  
  kind TEXT NOT NULL CHECK (kind IN ('commit','working_tree','staged','patch')),  
  label TEXT,  
  base_commit_sha TEXT,  
  workspace_hash TEXT NOT NULL,  
  parent_snapshot_id INTEGER REFERENCES repository_snapshots(id) ON DELETE SET NULL,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  pinned INTEGER NOT NULL DEFAULT 0 CHECK (pinned IN (0,1)),  
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN ('active','pruned')),  
  UNIQUE (repo_id, kind, workspace_hash)  
);
```

```
CREATE INDEX idx_snapshots_repo_created ON repository_snapshots(repo_id,  
created_at);
```

```
CREATE TABLE file_entities (  
  id INTEGER PRIMARY KEY,  
  repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
```



```
file_key TEXT NOT NULL,  
origin_identity TEXT NOT NULL,  
collision_index INTEGER NOT NULL DEFAULT 0,  
created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
UNIQUE (repo_id, file_key),  
UNIQUE (repo_id, origin_identity)  
);
```

```
CREATE TABLE snapshot_files (  
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE  
    CASCADE,  
    file_id INTEGER NOT NULL REFERENCES file_entities(id) ON DELETE RESTRICT,  
    path TEXT NOT NULL,  
    normalized_path TEXT NOT NULL,  
    package_name TEXT,  
    language TEXT,  
    content_hash TEXT NOT NULL,  
    size_bytes INTEGER NOT NULL DEFAULT 0,  
    is_generated INTEGER NOT NULL DEFAULT 0 CHECK (is_generated IN (0,1)),  
    is_binary INTEGER NOT NULL DEFAULT 0 CHECK (is_binary IN (0,1)),  
    PRIMARY KEY (snapshot_id, file_id),  
    UNIQUE (snapshot_id, normalized_path)  
);
```

```
CREATE INDEX idx_snapshot_files_path ON snapshot_files(snapshot_id,  
normalized_path);  
CREATE INDEX idx_snapshot_files_package ON snapshot_files(snapshot_id,  
package_name);
```

```
CREATE TABLE node_entities (  
    id INTEGER PRIMARY KEY,  
    repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,  
    entity_key TEXT NOT NULL,  
    canonical_identity TEXT NOT NULL,  
    collision_index INTEGER NOT NULL DEFAULT 0,  
    kind TEXT NOT NULL CHECK (kind IN (  
        'package','file','function','method','class','interface','type','route','test',  
        'adr','doc_section','external_dep','unresolved'  
    )),  
    qualified_name TEXT NOT NULL,  
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (repo_id, entity_key),  
    UNIQUE (repo_id, canonical_identity)
```

);

CREATE INDEX idx_node_entities_name **ON** node_entities(repo_id, qualified_name);

CREATE INDEX idx_node_entities_kind **ON** node_entities(repo_id, kind);

CREATE TABLE snapshot_nodes (
 snapshot_id **INTEGER NOT NULL**,
 node_id **INTEGER NOT NULL REFERENCES** node_entities(id) **ON DELETE RESTRICT**,
 file_id **INTEGER**,
 display_name **TEXT NOT NULL**,
 span_start **INTEGER**,
 span_end **INTEGER**,
 line_start **INTEGER**,
 line_end **INTEGER**,
 signature **TEXT**,
 body_hash **TEXT**,
 exported **INTEGER NOT NULL DEFAULT 0 CHECK** (exported **IN** (0,1)),
 analyzer_version **TEXT NOT NULL**,
 PRIMARY KEY (snapshot_id, node_id),
 FOREIGN KEY (snapshot_id) **REFERENCES** repository_snapshots(id) **ON DELETE CASCADE**,
 FOREIGN KEY (snapshot_id, file_id) **REFERENCES** snapshot_files(snapshot_id, file_id) **ON DELETE CASCADE**,
 CHECK ((span_start **IS NULL** **AND** span_end **IS NULL**) **OR**
 (span_start **IS NOT NULL** **AND** span_end **IS NOT NULL** **AND** span_end >= span_start))
);

CREATE INDEX idx_snapshot_nodes_file **ON** snapshot_nodes(snapshot_id, file_id);

CREATE INDEX idx_snapshot_nodes_lines **ON** snapshot_nodes(snapshot_id, file_id, line_start, line_end);

CREATE TABLE edge_entities (
 id **INTEGER PRIMARY KEY**,
 repo_id **INTEGER NOT NULL REFERENCES** repositories(id) **ON DELETE CASCADE**,
 entity_key **TEXT NOT NULL**,
 canonical_identity **TEXT NOT NULL**,
 collision_index **INTEGER NOT NULL DEFAULT 0**,
 src_node_id **INTEGER NOT NULL REFERENCES** node_entities(id) **ON DELETE RESTRICT**,
 dst_node_id **INTEGER NOT NULL REFERENCES** node_entities(id) **ON DELETE RESTRICT**,
 relation **TEXT NOT NULL CHECK** (relation **IN** (

```

        'contains','imports','exports','references','calls','implements','extends',
        'tests','routes_to','documents','changed_with'
    )),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (repo_id, entity_key),
    UNIQUE (repo_id, canonical_identity)
);

CREATE INDEX idx_edge_entities_src ON edge_entities(src_node_id, relation);
CREATE INDEX idx_edge_entities_dst ON edge_entities(dst_node_id, relation);

CREATE TABLE snapshot_edges (
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
    CASCADE,
    edge_id INTEGER NOT NULL REFERENCES edge_entities(id) ON DELETE RESTRICT,
    origin TEXT NOT NULL CHECK (origin IN
('compiler','heuristic','git','doc','human','llm')),
    confidence TEXT NOT NULL CHECK (confidence IN ('certain','likely','inferred')),
    resolution TEXT NOT NULL CHECK (resolution IN ('resolved','partial','unresolved')),
    analyzer_version TEXT NOT NULL,
    PRIMARY KEY (snapshot_id, edge_id)
);

CREATE INDEX idx_snapshot_edges_snapshot ON snapshot_edges(snapshot_id, edge_id);
CREATE INDEX idx_snapshot_edges_origin ON snapshot_edges(snapshot_id, origin,
confidence, resolution);

CREATE TABLE evidence_items (
    id INTEGER PRIMARY KEY,
    snapshot_id INTEGER NOT NULL,
    file_id INTEGER NOT NULL,
    evidence_kind TEXT NOT NULL CHECK (evidence_kind IN (
        'source','documentation','git','human_annotation','tool_event'
    )),
    line_start INTEGER,
    line_end INTEGER,
    column_start INTEGER,
    column_end INTEGER,
    commit_sha TEXT,
    excerpt_hash TEXT,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (snapshot_id, file_id) REFERENCES snapshot_files(snapshot_id, file_id)
    ON DELETE CASCADE,

```

```
    CHECK ((line_start IS NULL AND line_end IS NULL) OR
           (line_start IS NOT NULL AND line_end IS NOT NULL AND line_end >= line_start))
);
```

```
CREATE TABLE node_evidence (
  snapshot_id INTEGER NOT NULL,
  node_id INTEGER NOT NULL,
  evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
  PRIMARY KEY (snapshot_id, node_id, evidence_id),
  FOREIGN KEY (snapshot_id, node_id) REFERENCES snapshot_nodes(snapshot_id,
node_id) ON DELETE CASCADE
);
```

```
CREATE TABLE edge_evidence (
  snapshot_id INTEGER NOT NULL,
  edge_id INTEGER NOT NULL,
  evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
  PRIMARY KEY (snapshot_id, edge_id, evidence_id),
  FOREIGN KEY (snapshot_id, edge_id) REFERENCES snapshot_edges(snapshot_id,
edge_id) ON DELETE CASCADE
);
```

```
INSERT INTO schema_migrations(version) VALUES (1);
COMMIT;
```

1.10.2 Migration 002 - boundaries and explicit decisions

```
PRAGMA foreign_keys = ON;
BEGIN IMMEDIATE;
```

```
CREATE TABLE boundary_entities (
  id INTEGER PRIMARY KEY,
  repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
  boundary_key TEXT NOT NULL,
  name TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
  UNIQUE (repo_id, boundary_key),
  UNIQUE (repo_id, name)
);
```

```
CREATE TABLE snapshot_boundaries (
  snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
```

```

CASCADE,
    boundary_id INTEGER NOT NULL REFERENCES boundary_entities(id) ON DELETE
RESTRICT,
    rule_kind TEXT NOT NULL CHECK (rule_kind IN ('allow','deny','layer','ownership')),
    rule_json TEXT NOT NULL CHECK (json_valid(rule_json)),
    origin TEXT NOT NULL CHECK (origin IN ('config','documentation','human')),
    confidence TEXT NOT NULL CHECK (confidence IN ('certain','likely')),
    source_file_id INTEGER,
    PRIMARY KEY (snapshot_id, boundary_id),
    FOREIGN KEY (snapshot_id, source_file_id) REFERENCES snapshot_files(snapshot_id,
file_id) ON DELETE SET NULL
);

```

```

CREATE TABLE boundary_evidence (
    snapshot_id INTEGER NOT NULL,
    boundary_id INTEGER NOT NULL,
    evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
    PRIMARY KEY (snapshot_id, boundary_id, evidence_id),
    FOREIGN KEY (snapshot_id, boundary_id) REFERENCES
snapshot_boundaries(snapshot_id, boundary_id) ON DELETE CASCADE
);

```

```

CREATE TABLE decision_entities (
    id INTEGER PRIMARY KEY,
    repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,
    decision_key TEXT NOT NULL,
    kind TEXT NOT NULL CHECK (kind IN ('adr','doc','annotation')),
    title TEXT NOT NULL,
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (repo_id, decision_key)
);

```

```

CREATE TABLE snapshot_decisions (
    snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE
CASCADE,
    decision_id INTEGER NOT NULL REFERENCES decision_entities(id) ON DELETE
RESTRICT,
    status TEXT NOT NULL CHECK (status IN
('proposed','accepted','deprecated','superseded','unknown')),
    source_file_id INTEGER,
    body_excerpt TEXT,
    content_hash TEXT,

```

```
    PRIMARY KEY (snapshot_id, decision_id),
    FOREIGN KEY (snapshot_id, source_file_id) REFERENCES snapshot_files(snapshot_id,
file_id) ON DELETE SET NULL
);
```

```
CREATE TABLE decision_evidence (
    snapshot_id INTEGER NOT NULL,
    decision_id INTEGER NOT NULL,
    evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
    PRIMARY KEY (snapshot_id, decision_id, evidence_id),
    FOREIGN KEY (snapshot_id, decision_id) REFERENCES
snapshot_decisions(snapshot_id, decision_id) ON DELETE CASCADE
);
```

```
CREATE TABLE decision_links (
    id INTEGER PRIMARY KEY,
    snapshot_id INTEGER NOT NULL,
    decision_id INTEGER NOT NULL,
    node_id INTEGER,
    edge_id INTEGER,
    boundary_id INTEGER,
    confidence TEXT NOT NULL CHECK (confidence IN ('certain','likely')),
    evidence_id INTEGER NOT NULL REFERENCES evidence_items(id) ON DELETE
CASCADE,
    FOREIGN KEY (snapshot_id, decision_id) REFERENCES
snapshot_decisions(snapshot_id, decision_id) ON DELETE CASCADE,
    FOREIGN KEY (snapshot_id, node_id) REFERENCES snapshot_nodes(snapshot_id,
node_id) ON DELETE CASCADE,
    FOREIGN KEY (snapshot_id, edge_id) REFERENCES snapshot_edges(snapshot_id,
edge_id) ON DELETE CASCADE,
    FOREIGN KEY (snapshot_id, boundary_id) REFERENCES
snapshot_boundaries(snapshot_id, boundary_id) ON DELETE CASCADE,
    CHECK ((node_id IS NOT NULL) + (edge_id IS NOT NULL) + (boundary_id IS NOT
NULL) = 1)
);
```

```
CREATE UNIQUE INDEX idx_decision_link_node ON decision_links(snapshot_id,
decision_id, node_id) WHERE node_id IS NOT NULL;
CREATE UNIQUE INDEX idx_decision_link_edge ON decision_links(snapshot_id,
decision_id, edge_id) WHERE edge_id IS NOT NULL;
CREATE UNIQUE INDEX idx_decision_link_boundary ON decision_links(snapshot_id,
decision_id, boundary_id) WHERE boundary_id IS NOT NULL;
```

```
INSERT INTO schema_migrations(version) VALUES (2);  
COMMIT;
```

1.10.3 Migration 003 - tasks, retrieval, observed events, changes, tests

```
PRAGMA foreign_keys = ON;  
BEGIN IMMEDIATE;
```

```
CREATE TABLE tasks (  
  id INTEGER PRIMARY KEY,  
  repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,  
  base_snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON  
DELETE RESTRICT,  
  agent TEXT NOT NULL,  
  description TEXT NOT NULL,  
  status TEXT NOT NULL DEFAULT 'active' CHECK (status IN  
('active','completed','aborted','deleted')),  
  observation_coverage TEXT NOT NULL DEFAULT 'partial' CHECK (  
    observation_coverage IN ('complete\_for\_registered\_sources','partial','unknown')  
  ),  
  started_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  ended_at TEXT  
);
```

```
CREATE INDEX idx_tasks_repo_started ON tasks(repo_id, started_at);
```

```
CREATE TABLE retrieval_events (  
  id INTEGER PRIMARY KEY,  
  task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,  
  snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE  
RESTRICT,  
  tool TEXT NOT NULL CHECK (tool IN (  
    'repo\_overview','find\_symbol','symbol\_context','find\_tests','impact','path'  
  )),  
  args_json TEXT NOT NULL CHECK (json_valid(args_json)),  
  requested_token_budget INTEGER,  
  estimated_response_tokens INTEGER,  
  truncated INTEGER NOT NULL DEFAULT 0 CHECK (truncated IN (0,1)),  
  next_cursor TEXT,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE INDEX idx_retrieval_events_task ON retrieval_events(task_id, created_at);
```

```

CREATE TABLE retrieval_result_nodes (
  event_id INTEGER NOT NULL REFERENCES retrieval_events(id) ON DELETE
CASCADE,
  node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE RESTRICT,
  rank_position INTEGER NOT NULL,
  score REAL,
  representation TEXT NOT NULL CHECK (representation IN
('body','signature','name','aggregate')),
  stale INTEGER NOT NULL DEFAULT 0 CHECK (stale IN (0,1)),
  PRIMARY KEY (event_id, node_id)
);

```

```

CREATE TABLE retrieval_result_edges (
  event_id INTEGER NOT NULL REFERENCES retrieval_events(id) ON DELETE
CASCADE,
  edge_id INTEGER NOT NULL REFERENCES edge_entities(id) ON DELETE RESTRICT,
  rank_position INTEGER NOT NULL,
  score REAL,
  stale INTEGER NOT NULL DEFAULT 0 CHECK (stale IN (0,1)),
  PRIMARY KEY (event_id, edge_id)
);

```

```

CREATE TABLE retrieval_omissions (
  id INTEGER PRIMARY KEY,
  event_id INTEGER NOT NULL REFERENCES retrieval_events(id) ON DELETE
CASCADE,
  target_kind TEXT NOT NULL CHECK (target_kind IN ('node','edge')),
  node_id INTEGER REFERENCES node_entities(id) ON DELETE RESTRICT,
  edge_id INTEGER REFERENCES edge_entities(id) ON DELETE RESTRICT,
  rank_position INTEGER NOT NULL,
  score REAL,
  reason TEXT NOT NULL,
  CHECK ((target_kind = 'node' AND node_id IS NOT NULL AND edge_id IS NULL) OR
(target_kind = 'edge' AND edge_id IS NOT NULL AND node_id IS NULL))
);

```

```

CREATE TABLE agent_events (
  id INTEGER PRIMARY KEY,
  task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
  snapshot_id INTEGER REFERENCES repository_snapshots(id) ON DELETE RESTRICT,
  event_type TEXT NOT NULL CHECK (event_type IN (

```

```

'file_read_observed','plan_mentioned','modified','test_selected','test_executed','capture_int

```



```

errupted'
    )),
    source TEXT NOT NULL CHECK (source IN
('claude_hook','codex_log','transcript','manual')),
    payload_json TEXT CHECK (payload_json IS NULL OR json_valid(payload_json)),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE agent_event_targets (
    id INTEGER PRIMARY KEY,
    event_id INTEGER NOT NULL REFERENCES agent_events(id) ON DELETE CASCADE,
    target_kind TEXT NOT NULL CHECK (target_kind IN ('file','node')),
    file_id INTEGER REFERENCES file_entities(id) ON DELETE RESTRICT,
    node_id INTEGER REFERENCES node_entities(id) ON DELETE RESTRICT,
    CHECK ((target_kind = 'file' AND file_id IS NOT NULL AND node_id IS NULL) OR
        (target_kind = 'node' AND node_id IS NOT NULL AND file_id IS NULL))
);

CREATE UNIQUE INDEX idx_agent_event_target_file
    ON agent_event_targets(event_id, file_id)
    WHERE target_kind = 'file' AND file_id IS NOT NULL;

CREATE UNIQUE INDEX idx_agent_event_target_node
    ON agent_event_targets(event_id, node_id)
    WHERE target_kind = 'node' AND node_id IS NOT NULL;

CREATE TABLE change_sets (
    id INTEGER PRIMARY KEY,
    task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,
    base_snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON
DELETE RESTRICT,
    head_snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON
DELETE RESTRICT,
    kind TEXT NOT NULL CHECK (kind IN ('working_tree','staged','commit','patch')),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CHECK (base_snapshot_id <> head_snapshot_id)
);

CREATE TABLE change_set_files (
    change_set_id INTEGER NOT NULL REFERENCES change_sets(id) ON DELETE
CASCADE,
    file_id INTEGER NOT NULL REFERENCES file_entities(id) ON DELETE RESTRICT,
    change_kind TEXT NOT NULL CHECK (change_kind IN

```

```
('added','modified','deleted','renamed')),  
  before_hash TEXT,  
  after_hash TEXT,  
  old_path TEXT,  
  new_path TEXT,  
  PRIMARY KEY (change_set_id, file_id)  
);
```

```
CREATE TABLE test_runs (  
  id INTEGER PRIMARY KEY,  
  task_id INTEGER NOT NULL REFERENCES tasks(id) ON DELETE CASCADE,  
  snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE  
RESTRICT,  
  command_redacted TEXT NOT NULL,  
  status TEXT NOT NULL CHECK (status IN  
(  
'running','passed','failed','cancelled','unknown'  
)),  
  exit_code INTEGER,  
  started_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  ended_at TEXT,  
  output_retained INTEGER NOT NULL DEFAULT 0 CHECK (output_retained IN (0,1))  
);
```

```
CREATE TABLE test_run_cases (  
  id INTEGER PRIMARY KEY,  
  test_run_id INTEGER NOT NULL REFERENCES test_runs(id) ON DELETE CASCADE,  
  test_node_id INTEGER REFERENCES node_entities(id) ON DELETE SET NULL,  
  test_name TEXT NOT NULL,  
  status TEXT NOT NULL CHECK (status IN ('passed','failed','skipped','unknown')),  
  duration_ms INTEGER  
);
```

```
INSERT INTO schema_migrations(version) VALUES (3);  
COMMIT;
```

1.10.4 Migration 004 - layouts and quarantined summaries

```
PRAGMA foreign_keys = ON;  
BEGIN IMMEDIATE;
```

```
CREATE TABLE layout_positions (  
  repo_id INTEGER NOT NULL REFERENCES repositories(id) ON DELETE CASCADE,  
  abstraction_level TEXT NOT NULL CHECK (abstraction_level IN  
(  
'package','file','symbol'  
)),  
  view_key TEXT NOT NULL DEFAULT 'base',
```

```
node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE CASCADE,  
x REAL NOT NULL,  
y REAL NOT NULL,  
z REAL NOT NULL DEFAULT 0,  
pinned INTEGER NOT NULL DEFAULT 0 CHECK (pinned IN (0,1)),  
anchor_group TEXT,  
layout_version INTEGER NOT NULL DEFAULT 1,  
last_snapshot_id INTEGER REFERENCES repository_snapshots(id) ON DELETE SET  
NULL,  
updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
PRIMARY KEY (repo_id, abstraction_level, view_key, node_id)  
);
```

```
CREATE INDEX idx_layout_level ON layout_positions(repo_id, abstraction_level,  
view_key);
```

```
CREATE TABLE summaries (  
  id INTEGER PRIMARY KEY,  
  snapshot_id INTEGER NOT NULL REFERENCES repository_snapshots(id) ON DELETE  
CASCADE,  
  node_id INTEGER NOT NULL REFERENCES node_entities(id) ON DELETE CASCADE,  
  model TEXT NOT NULL,  
  prompt_hash TEXT NOT NULL,  
  summary_text TEXT NOT NULL,  
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE (snapshot_id, node_id, model, prompt_hash)  
);
```

```
INSERT INTO schema_migrations(version) VALUES (4);  
COMMIT;
```

1.10.5 Migration 005 - full-text search

```
PRAGMA foreign_keys = ON;  
BEGIN IMMEDIATE;
```

```
CREATE VIRTUAL TABLE node_fts USING fts5(  
  snapshot_id UNINDEXED,  
  node_id UNINDEXED,  
  display_name,  
  qualified_name,  
  signature,  
  path,  
  tokenize = 'unicode61'
```

);

```
INSERT INTO schema_migrations(version) VALUES (5);  
COMMIT;
```

1.11 10. Required snapshot validation

The schema does not enforce that each snapshot edge's endpoint nodes are members of the same snapshot. Run this after every completed snapshot build:

```
SELECT  
  se.snapshot_id,  
  se.edge_id,  
  ee.src_node_id,  
  ee.dst_node_id,  
  CASE WHEN src.node_id IS NULL THEN 1 ELSE 0 END AS  
missing_source_membership,  
  CASE WHEN dst.node_id IS NULL THEN 1 ELSE 0 END AS  
missing_destination_membership  
FROM snapshot_edges AS se  
JOIN edge_entities AS ee ON ee.id = se.edge_id  
LEFT JOIN snapshot_nodes AS src  
  ON src.snapshot_id = se.snapshot_id AND src.node_id = ee.src_node_id  
LEFT JOIN snapshot_nodes AS dst  
  ON dst.snapshot_id = se.snapshot_id AND dst.node_id = ee.dst_node_id  
WHERE src.node_id IS NULL OR dst.node_id IS NULL;
```

A valid snapshot returns zero rows. Invalid snapshots are not served through MCP and do not replace the last valid active snapshot.

1.12 11. Three-way extracted-edge diff

A complete graph diff includes added edges, removed edges, and membership-property changes on an edge that exists in both snapshots.

```
WITH edge_diff AS (  
  SELECT  
    'added' AS change_kind,  
    head.edge_id,  
    NULL AS before_origin,  
    NULL AS before_confidence,  
    NULL AS before_resolution,  
    head.origin AS after_origin,  
    head.confidence AS after_confidence,  
    head.resolution AS after_resolution  
  FROM snapshot_edges AS head
```

```

WHERE head.snapshot_id = :head_snapshot
AND NOT EXISTS (
    SELECT 1 FROM snapshot_edges AS base
    WHERE base.snapshot_id = :base_snapshot
    AND base.edge_id = head.edge_id
)

```

UNION ALL

```

SELECT
    'removed',
    base.edge_id,
    base.origin,
    base.confidence,
    base.resolution,
    NULL,
    NULL,
    NULL
FROM snapshot_edges AS base
WHERE base.snapshot_id = :base_snapshot
AND NOT EXISTS (
    SELECT 1 FROM snapshot_edges AS head
    WHERE head.snapshot_id = :head_snapshot
    AND head.edge_id = base.edge_id
)

```

UNION ALL

```

SELECT
    'resolution_or_provenance_changed',
    head.edge_id,
    base.origin,
    base.confidence,
    base.resolution,
    head.origin,
    head.confidence,
    head.resolution
FROM snapshot_edges AS base
JOIN snapshot_edges AS head
    ON head.edge_id = base.edge_id
    AND head.snapshot_id = :head_snapshot
WHERE base.snapshot_id = :base_snapshot
AND (

```

```

        base.origin <> head.origin OR
        base.confidence <> head.confidence OR
        base.resolution <> head.resolution
    )
)
SELECT
    d.change_kind,
    src.qualified_name AS source,
    e.relation,
    dst.qualified_name AS destination,
    d.before_origin,
    d.before_confidence,
    d.before_resolution,
    d.after_origin,
    d.after_confidence,
    d.after_resolution
FROM edge_diff AS d
JOIN edge_entities AS e ON e.id = d.edge_id
JOIN node_entities AS src ON src.id = e.src_node_id
JOIN node_entities AS dst ON dst.id = e.dst_node_id;

```

1.13 12. Rename and move coalescing

Raw entity identity remains unchanged. Review mode adds a presentation-time coalescing pass.

1.13.1 Stage A - likely move with name preserved

Pair one removed node with one added node when all match:

- kind;
- unqualified name;
- body hash;
- analyzer version.

1.13.2 Stage B - likely symbol rename

For unmatched nodes, pair one removed node with one added node when all match:

- kind;
- body hash;
- analyzer version;
- unqualified names differ.

Recursive functions and symbols whose bodies contain self-references may not Stage-B match after rename because the body text changes with the name. The safe fallback is the raw added/removed diff.

1.13.3 Ambiguity

If multiple removed or added candidates share a candidate signature and no unique Git rename tie-breaker exists, do not coalesce automatically. Show **Possible move or rename - ambiguous** and preserve the raw rows.

1.13.4 Edges

After node pairs exist, pair removed and added edges only when:

- relation is identical;
- mapped source and destination endpoints match;
- exactly one corresponding added edge exists.

Show both raw and coalesced counts.

1.14 13. Foreign-key-safe garbage collection

Entity collection must respect all surviving ON DELETE RESTRICT references.

```
PRAGMA foreign_keys = ON;  
BEGIN IMMEDIATE;
```

```
DELETE FROM edge_entities  
WHERE NOT EXISTS (SELECT 1 FROM snapshot_edges WHERE snapshot_edges.edge_id  
= edge_entities.id)  
AND NOT EXISTS (SELECT 1 FROM retrieval_result_edges WHERE  
retrieval_result_edges.edge_id = edge_entities.id)  
AND NOT EXISTS (SELECT 1 FROM retrieval_omissions WHERE  
retrieval_omissions.edge_id = edge_entities.id)  
AND NOT EXISTS (SELECT 1 FROM decision_links WHERE decision_links.edge_id =  
edge_entities.id);
```

```
DELETE FROM node_entities  
WHERE NOT EXISTS (SELECT 1 FROM snapshot_nodes WHERE  
snapshot_nodes.node_id = node_entities.id)  
AND NOT EXISTS (SELECT 1 FROM retrieval_result_nodes WHERE  
retrieval_result_nodes.node_id = node_entities.id)  
AND NOT EXISTS (SELECT 1 FROM retrieval_omissions WHERE  
retrieval_omissions.node_id = node_entities.id)  
AND NOT EXISTS (SELECT 1 FROM agent_event_targets WHERE  
agent_event_targets.node_id = node_entities.id)  
AND NOT EXISTS (SELECT 1 FROM decision_links WHERE decision_links.node_id =
```

```

node_entities.id)
AND NOT EXISTS (
    SELECT 1 FROM edge_entities
    WHERE edge_entities.src_node_id = node_entities.id
    OR edge_entities.dst_node_id = node_entities.id
);

DELETE FROM file_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_files WHERE snapshot_files.file_id =
file_entities.id)
AND NOT EXISTS (SELECT 1 FROM agent_event_targets WHERE
agent_event_targets.file_id = file_entities.id)
AND NOT EXISTS (SELECT 1 FROM change_set_files WHERE change_set_files.file_id =
file_entities.id);

DELETE FROM boundary_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_boundaries WHERE
snapshot_boundaries.boundary_id = boundary_entities.id)
AND NOT EXISTS (SELECT 1 FROM decision_links WHERE decision_links.boundary_id
= boundary_entities.id);

DELETE FROM decision_entities
WHERE NOT EXISTS (SELECT 1 FROM snapshot_decisions WHERE
snapshot_decisions.decision_id = decision_entities.id);

COMMIT;
PRAGMA foreign_key_check;

```

1.15 14. Updated twelve-week gates

1.15.1 Weeks 1-2

- migrations run on an empty SQLite database;
- commit and working-tree snapshots coexist;
- foreign_key_check returns zero rows;
- dangling endpoint memberships equal zero;
- 150k LOC initial index under 60 seconds on the target machine;
- import/export fixture gates pass.

1.15.2 Week 3

- relation-specific fixture recall reported;
- real-repository precision samples collected;
- unresolved call rate reported;
- unsupported relations disabled rather than mislabeled.

1.15.3 Week 4

- exactly six MCP tools;
- every call logged;
- hook interruption changes observation coverage to partial.

1.15.4 Week 5

- hard includes and omission manifests;
- 100% of truncated responses include an omission manifest;
- no direct compiler-certain caller/callee is silently dropped.

1.15.5 Week 6

Latency gates:

- single-file change: under 2 seconds p95;
- package-scope invalidation: under 10 seconds p95;
- dependency-invalidated region: best effort with measured, documented ceiling and stale flags retained until completion.

1.15.6 Weeks 7-8

- package map interactive in under 5 seconds for 250k LOC;
- file expansion does not move unrelated packages;
- symbol expansion does not load the full symbol graph;
- aggregate coverage reconciles with child observations.

1.15.7 Week 9

- working-tree comparison works without a commit;
- fixture diff recall and sampled real-diff precision meet relation targets;
- UI distinguishes extracted graph differences from runtime architecture truth.

1.15.8 Week 10

- 2D, 2.5D, and 3D consume the same entity and overlay query results;
- every depth field maps to a named field;
- this remains the first work cut if the schedule slips.

1.15.9 Week 11

Run the agent benchmark against plain Claude Code and a real existing graph-MCP baseline. Report results regardless of outcome.

1.15.10 Week 12

Do not attempt the full human study. Freeze the prototype, run an internal visual-task dry run, finalize the pilot protocol, verify privacy behavior, publish benchmark results, and record the demo.

1.16 15. Failure conditions

- If graph retrieval lowers task success or produces no improvement in unrelated edits, missed consumers, unsupported claims, or evidence quality, remove it as a product claim.
- If reviewers ignore the attention overlay or it creates overconfidence, retain a table and diff but drop the map.
- If decision links remain inaccurate or unused, reduce them to a simple documentation-reference panel.
- If 3D misses the predefined useful-effect criteria, delete it from the product branch and report the negative result.
- Stop entirely if graph quality remains untrustworthy or interactive indexing/stale-state handling cannot be made reliable.

1.17 16. Frozen-spec note

Tadori v2.1 was accepted after independent execution checks confirmed:

- migrations build on an empty SQLite database;
- the worked snapshot example inserts cleanly;
- the three-way diff returns added, removed, and resolution/provenance-change rows;
- dangling endpoint validation returns zero;
- foreign_key_check returns zero rows;
- pipe-delimited canonical SHA-256 identities match;
- Wilson table calculations are attainable and consistent.

No further specification revision should occur without a new defect report.